

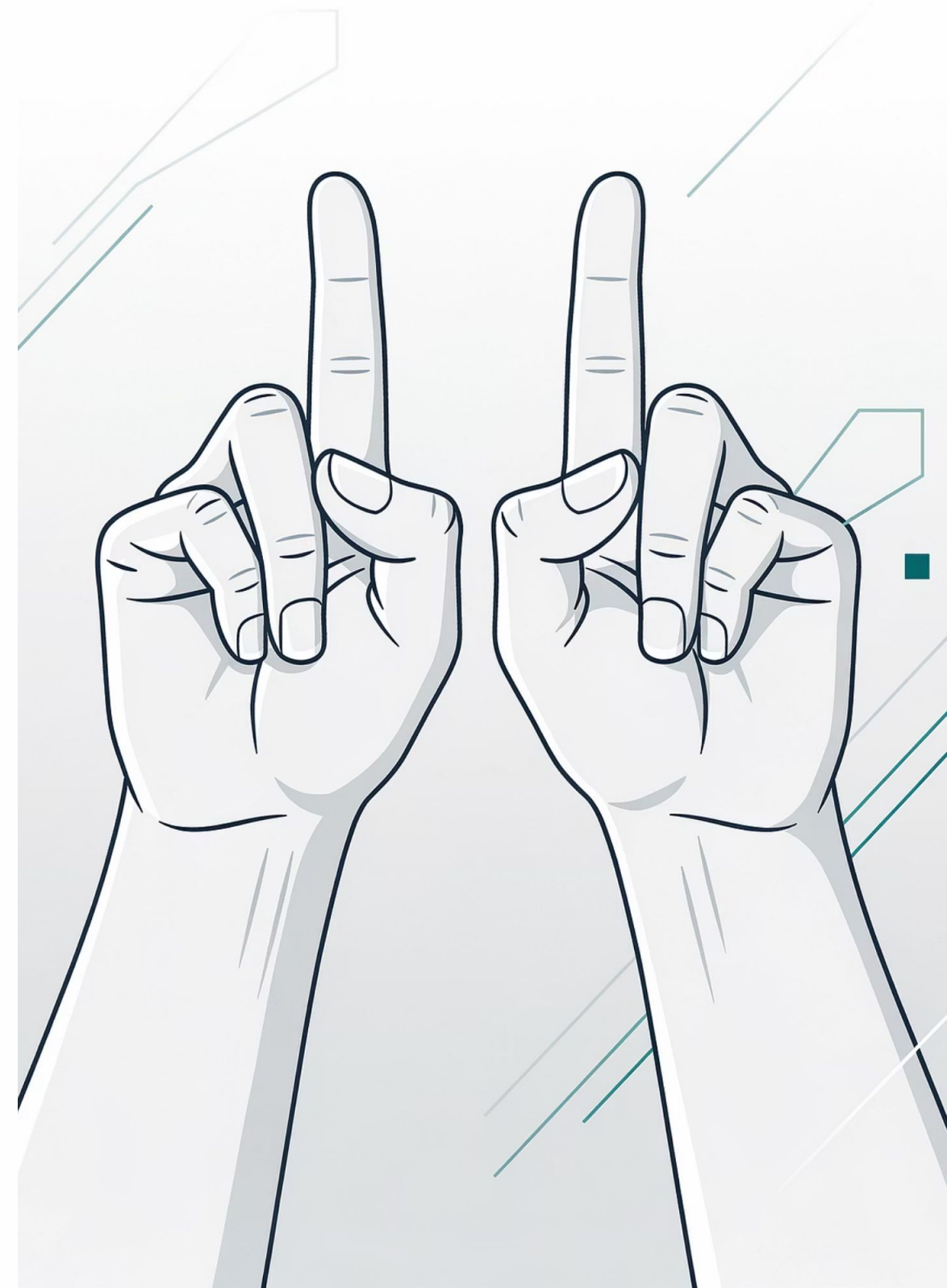
Real-Time ASL Translation

Keypoint-Based Transformer for Sign Language Recognition on CPU

Radhika Khurana · Jian Gao · Hrishikesh Pradhan · Gyula Planky

Applied Deep Learning · Northeastern University · Spring 2026

github.com/khuranaradhika/asl-realti



The Problem: 70 Million People. No Accessible AI.

State-of-the-art video models like I3D, SlowFast, and VideoMAE achieve impressive accuracy — but only on expensive GPUs. A single raw video frame contains **6.2 million pixels** to process. These models cannot run on a laptop, a phone, or a kiosk, and their high latency makes real-time conversation impossible. No accessible, deployable solution exists today.

GPU Dependency

Top models require dedicated NVIDIA hardware — unavailable to most end users

6.2M Pixels / Frame

Raw video is computationally expensive and privacy-invasive by design

High Latency

Processing delays destroy conversational flow — real-time is non-negotiable

No Deployment Path

The gap between research accuracy and real-world use remains unsolved



Our Insight: The Pixels Don't Matter. The Hands Do.

A human signs with their hands — not with the background, lighting, or camera angle. MediaPipe extracts **21 landmarks per hand × 3 coordinates (x, y, z) = 126 numbers per frame**. That is 126 floats versus 6,220,800 pixels — a **49,371× reduction** in data volume.

No raw video is stored at any point, making the system **privacy-preserving by design**. The result: the same semantic information at a fraction of the computational cost.

Raw Video

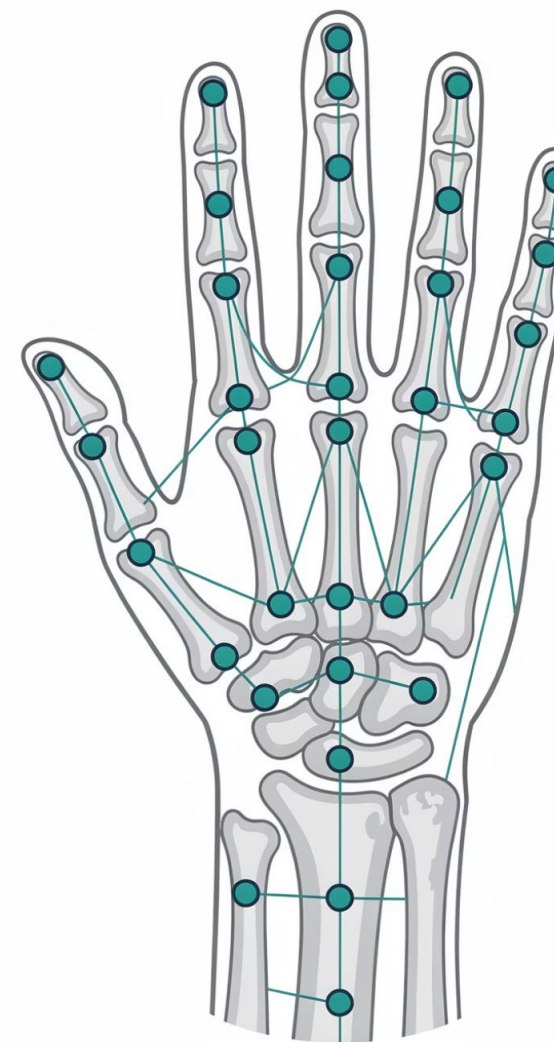
6,220,800 pixels/frame GPU

required · Privacy risk

Keypoints

126 floats/frame CPU-ready ·

Private by design



How MediaPipe Works: From Pixels to Numbers

MediaPipe HandLandmarker detects the hand in each frame and outputs **21 keypoints per hand**, each with three coordinates.

Coordinates are normalized relative to the image dimensions and the wrist joint, making detection **camera-agnostic** — it doesn't matter if you're on a MacBook webcam or a \$30 USB camera.

x, y — normalized pixel position (0.0–1.0)

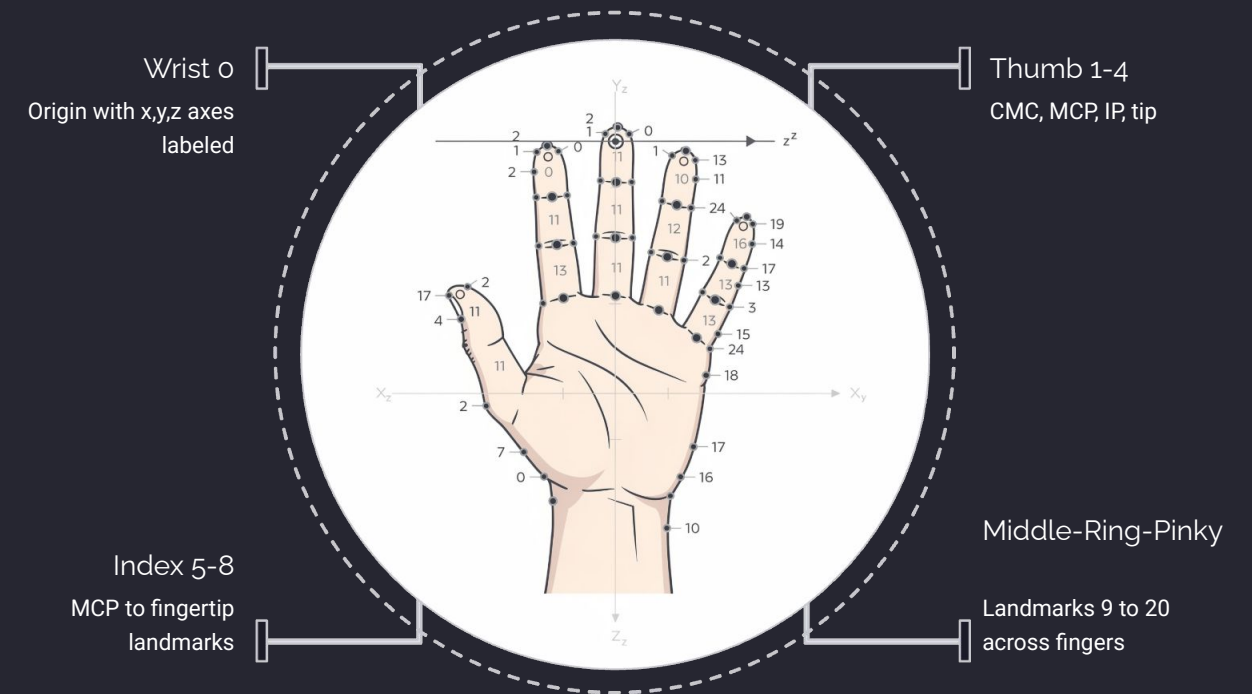
z — depth relative to wrist (estimated)

- Wrist-relative normalization removes scale variance
- Runs at 30fps on CPU in under 8ms per frame

i Normalization is what makes the model signer-agnostic. A large hand and a small hand produce the same landmark ratios for the same sign.

21 Landmark Points

Wrist · Thumb (4) · Index (4) · Middle (4) · Ring (4) · Pinky (4)



System Pipeline: Four Steps, Under 25ms



The entire inference chain — from live webcam frame to predicted ASL word — completes in under 25 milliseconds. The system exports to a **3MB ONNX file**, runs with no internet connection, and has been tested on an M1 MacBook, Windows laptop, and Raspberry Pi.

CPU Only No GPU required — runs on any modern consumer device	Offline Inference Zero network dependency at runtime — works anywhere
3MB Model ONNX export — embeddable in mobile apps, kiosks, browsers	<25ms Latency Real conversation speed — 8ms extraction + 15ms inference

Every Component, Every Latency, One Diagram



Stage 1: WEBCAM
30fps · 1920×1080 · RGB frames
Output: raw video frame (6,220,800 pixels)



Stage 2: MEDIAPIPE HANDLANDMARKER
Hand detection (palm detector)
Landmark regression (21 pts/hand)
Wrist-relative normalization
Hand-span scale normalization
Output: (126,) float32 vector
Latency: ~8ms on CPU
Model: hand_landmarker.task (25MB)



Stage 3: FRAME BUFFER
Rolling window of T=30 frames
Shape: (30, 126) = 3,780 floats
Padded to (150, 126) for model input
Updated every frame at 30fps



Stage 4: TRANSFORMER ENCODER (ONNX)
Linear: 126 → 128
Sinusoidal positional encoding
3× EncoderLayer (4 heads, FFN=256)
Global mean pool → (128,)
Linear: 128 → 1,896
Softmax → probability distribution
Parameters: 452,000 · Model file: 3.1MB ONNX
Latency: ~14ms on CPU



Stage 5: DECODER + DISPLAY
Top-3 predictions extracted
Confidence threshold: >30% to display
Smoothing: majority vote over 5 frames
Example output: FRIEND 78% · SISTER 18% · COLLEAGUE 4%

~8ms
MediaPipe
Latency

~14ms
Transformer
Latency

<1ms
Buffer
Latency

<1ms
Decode
Latency

<25ms
Total
End-to-end

Three Engineering Decisions Behind the Pipeline

Our design choices were driven by the need for real-time performance and accessibility on commodity hardware. Here's why we made them:

Why T=30 frames?

At 30fps, 30 frames = 1 second of signing.
Most ASL signs complete in 0.5–1.5 seconds. A 30-frame window captures a full sign without requiring precise segmentation. We pad shorter clips to T=150 with zeros — the attention mask prevents the model from attending to padding.

Why a padding mask?

The transformer would see padded zero frames as real data without a mask. We pass a boolean mask marking real vs padded frames. The global mean pool only averages over non-padded frames — preserving correct statistics regardless of clip length.

Why majority vote smoothing?

Frame-by-frame predictions oscillate — the model sees partial signs mid-signing. A 5-frame majority vote over consecutive predictions stabilizes output and prevents flicker. Only predictions above 30% confidence are displayed.

Data Engineering: Building the Dataset from Scratch

No single clean dataset existed. We aggregated three sources, handled dead links, batch-processed 108k videos overnight, and kept peak disk usage under 2GB at any moment through rolling deletion of raw video after keypoint extraction.

Dataset	Raw Videos	Extracted Signs	Classes	Status
WLASL	21,083 available	6,845 downloaded	2,000	70% of links dead since 2020
ASL Citizen	Pre-extracted	1,542 poses	~300	Google/HuggingFace — clean
Aslense	108,000	53,933	2,208	108k processed, raw deleted
Combined	—	52,998 train	1,896	This project



WLASL: 70% of hosting links were removed after 2020. Only 6,845 of 21,083 videos were recoverable.



Final keypoints: 2.7GB. Stratified val split: 5,567 samples across all 1,896 classes.

Why 1,896 Classes? The Vocabulary Density Tradeoff

The vocabulary size directly controls the difficulty of the learning problem. Too few classes and the model is useless. Too many and there are not enough samples per class to learn anything. We swept the full range to find the sweet spot.

Vocab	Samples/Class	Learnable?	Coverage
300	9.6	Barely	Limited
1,896	28.0	✓ Yes	Good
2,591	20.7	Marginal	Better
Full WLASL (1,861)	3.4	✗ No	—

Sweet Spot: 1,896 Classes

- All 1,896 classes have training data
- 96% of classes have 10+ samples
- 449 classes have 30+ samples
- 28 samples/class = minimum viable density



Choosing vocab=2,591 dropped accuracy by ~4 points in early epochs because samples/class fell below the learning threshold.

From 126 Numbers to a Predicted Sign

Step 1: Input

(Batch, Time, 126) · Each frame = 126 keypoint coordinates · T frames per signing clip, padded to 150

Step 4: × 3 Transformer Encoder Layers

Multi-head self-attention (4 heads) · FFN: $128 \rightarrow 256 \rightarrow 128$ · Pre-norm · Dropout: 0.1 · Residual connections

Step 7: CrossEntropy Loss

Label smoothing = 0.1 · Prevents overconfidence on ambiguous signs

Step 2: Linear Projection

$126 \rightarrow 128$ dimensions · Projects keypoints into model embedding space

Step 5: Global Mean Pool

Collapses (Batch, Time, 128) $\rightarrow$ (Batch, 128) · One vector, one label — this is why we use CE not CTC

Step 3: Sinusoidal Positional Encoding

Injects frame order information · sin/cos functions of position — no learned parameters

Step 6: Linear Classifier

$128 \rightarrow 1,896$ · One logit per sign class

Small Model vs Large Model: What Changes?

Metric	Small Model	Large Model
d_model	128	256
Layers	3	4
Attention heads	4	8
FFN dim	256	512
Parameters	452K	711K
Epoch time (MPS)	~70s	~150s
Best Top-1	~40%	44.0%

452K

Parameters

in the small model. Smaller than most image classifiers from 2012.

2X

Epoch Time Increase

going from small to large. 70s → 150s per epoch on Apple MPS.

+4pts

Top-1 Accuracy Gain

from small to large model. Worth the compute cost.

The whole thing is 452,000 parameters — smaller than most image classifiers from 2012.

Three Design Choices That Define the Architecture

Pre-norm vs Post-norm

We use Pre-norm (LayerNorm before attention, not after). Pre-norm training is more stable — gradients flow more cleanly through deep layers. Standard BERT uses post-norm; modern transformers (GPT, LLaMA) use pre-norm. We chose stability over convention.

Global Mean Pool vs CTC

Mean pooling collapses the temporal dimension by averaging all frame representations. This works because each clip contains exactly one sign — we don't need to align output tokens to input frames. CTC is designed for continuous sequences where you don't know where words start and end. Our problem doesn't have that ambiguity.

Why 4 heads, not 8?

With $d_{\text{model}}=128$, each attention head gets $128/4 = 32$ dimensions. Using 8 heads would give 16 dimensions per head — too small to learn meaningful attention patterns. Head dimension of 32 is the practical minimum for stable attention.

Signs Are Holistic. Attention Sees the Whole Thing.

1D CNN

Only sees 3 frames at a time. Misses the relationship between the start of the sign (hand position) and the end (finger configuration). Good for detecting local motion but blind to full-sign shape.
Top-1: 38.4%

BiLSTM

Sees frames sequentially — frame 1 influences frame 2 influences frame 3. By frame 30, information from frame 1 has passed through 29 hidden state updates and degraded. Long-range dependencies are lossy.
Top-1: 31.5%

☆ WINNER

Transformer ✓

Every frame attends to every other frame simultaneously. Frame 1 and frame 30 have a direct connection with no intermediaries. The model learns which frames are most important for distinguishing this sign from similar ones.
Top-1: 44.0%

❏ Why this matters for ASL: Many ASL signs are distinguished by their trajectory — not any single frame. WATER and WINE have similar hand shapes in isolation. The full motion path is what separates them. Self-attention captures the full trajectory relationship directly.

+18

Transformer vs BiLSTM
Points gained in Top-1 accuracy

+11

Transformer vs CNN
Points gained in Top-1 accuracy

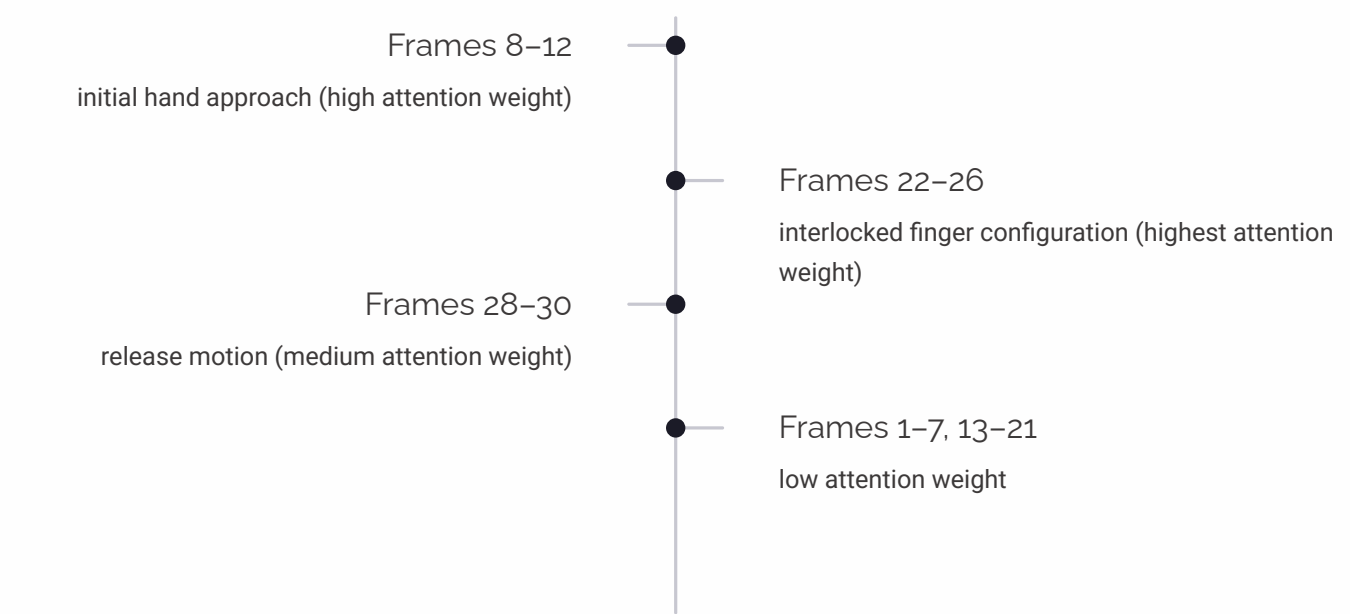
~450K

Parameter Count
All models share a similar parameter count

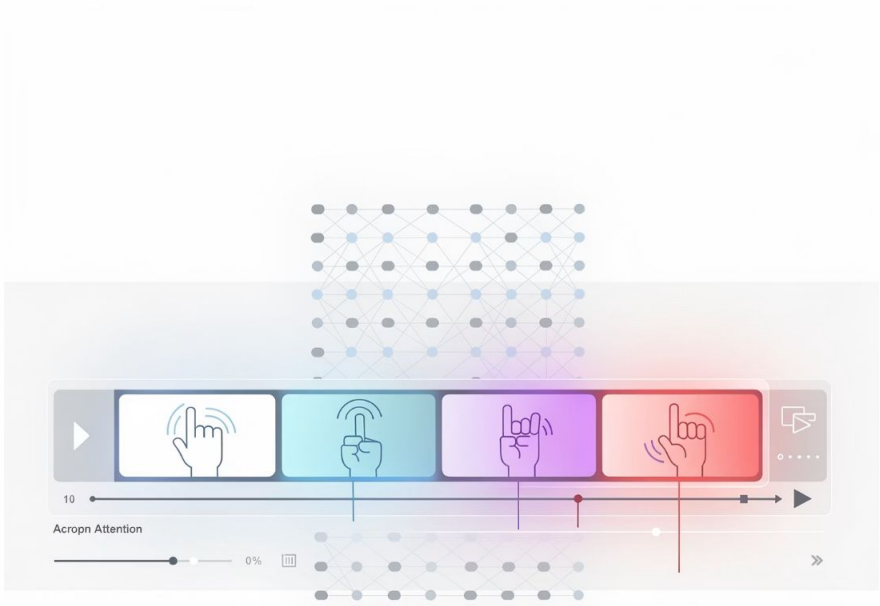
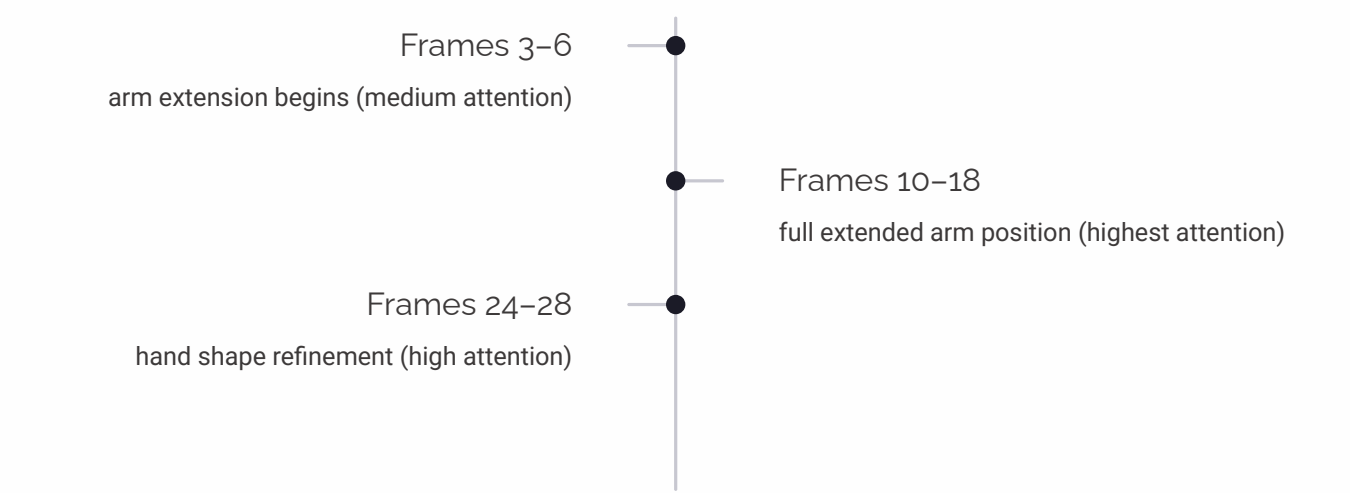
What Does the Model Actually Look At?

Attention weights reveal which frames are most discriminative – without being told.

Example 1: Predicting "FRIEND"



Example 2: Predicting "AIRPLANE"



The transformer's attention weights show which frames it relies on most when making a prediction. These are not hand-coded rules – the model discovered these temporal landmarks through training alone. For FRIEND, the model focuses on the interlocked finger configuration – the most visually distinctive moment. For AIRPLANE, it focuses on the extended arm – the defining feature of that sign. This is what makes attention powerful for sign language: signs have meaningful temporal landmarks that CNN windows miss and LSTM state degrades. Note: A `--visualize-attention` flag in `demo.py` can render these heatmaps live during inference.

□ The model has learned which moments in a signing motion are most discriminative – without being told. This is emergent temporal understanding.

Training Setup: Every Hyperparameter Is Justified

Component	Choice	Why
Loss	CrossEntropy + label smooth 0.1	Isolated word classification; prevents overconfidence. Not CTC — each clip contains exactly one sign.
Optimizer	AdamW lr=3e-4, wd=1e-4	Standard for transformers; weight decay regularizes without hurting convergence
Scheduler	Cosine annealing	Smooth LR decay over full training cycle
Sampler	WeightedRandomSampler	Equal class representation per batch — critical for 1,896-class imbalance
Grad Clip	max_norm=1.0	Prevents exploding gradients in early epochs
Augmentation	Flip, jitter, noise, normalize	Prevents overfitting to specific signers
Device	Apple M4 Pro MPS	~90–150s/epoch; no cloud GPU needed

❏ CrossEntropy is correct here — not CTC. CTC handles continuous sequence alignment. Mean pooling + CE is the correct formulation for single-sign clip classification.

The Augmentation Discovery: 67% Is Worse Than 44%

What the Numbers

Show

Model	Val Top-1	Converges	Real?
No augmentation	67.0%	Epoch 10	✗ Overfitting
With augmentation	44.0%	Epoch 150	✓ Generalizes



Without augmentation, the model memorizes exact hand trajectories from training signers. Val accuracy looks great — but it fails completely on a new signer in a live demo.

What Actually Happened

Without augmentation, the model sees **identical keypoint sequences every epoch**. It memorizes signer identity, not sign shape. The validation set shares the same recording conditions, so val accuracy appears high.

With **flip, jitter, and noise augmentation**, the model is forced to learn the geometric shape of a sign — invariant to hand size, camera position, and signer identity.



Augmentation is not optional. It is what makes the model deployable — not just accurate on a held-out set.

Experiment Journey: Six Experiments, Six Lessons

The most important lesson: **fixing the validation set** (stratified split) revealed the model had been learning all along. Measurement error masked a working system — accuracy jumped from 6.7% to 40.8% without touching the model.

Exp	Vocab	Samples	Val Samples	Top-1	Key Lesson
EXP-001	100	570	100	7.8%	Baseline established
EXP-002	100	570	100	3.9%	Small dataset + aug hurts
EXP-003	300	2,288	229	6.7%	Val set was broken
EXP-003 CE	300	2,288	456	40.8%	Fixed val → model was learning all along
EXP-004 Large	1,896	52,998	5,567	44.0%	Scale works
Distilled	1,896	52,998	5,567	49.5%	Student beats teacher

Ablation Study: Every Design Decision Validated

Model	Aug	WRS	Params	Top-1	Top-5	0% Classes
BiLSTM	✓	✓	735K	31.5%	53.2%	749
1D CNN	✓	✓	450K	38.4%	60.8%	612
Transformer (no aug)	✗	✗	452K	67%*	88%*	136
Transformer	✓	✗	452K	40.0%	56.0%	~500
Transformer Large	✓	✓	711K	44.0%	60.2%	465
Distilled Student	✓	✓	452K	49.5%	71.3%	374

* Overfitting — validation accuracy inflated by lack of augmentation. Not a real result.

Transformer > LSTM > CNN
Self-attention matters for full-duration temporal signing

WRS: +3 pts
Weighted sampling alone adds 3 percentage points

Larger Model: +4 pts
711K parameters over 452K yields +4 Top-1 accuracy

Distillation: +5.5 pts
Student beats teacher at 37% fewer parameters

Trained in PyTorch. Deployed Anywhere.

The Export Pipeline

Step 1: PyTorch Checkpoint (.pt)

transformer_d128_l3_v1896_combined_best.pt

658K parameters · training-mode · MPS only

Step 3:

`torch.onnx.export()` · sign_model.onnx · 3MB · framework-agnostic

Ops: MatMul, LayerNorm, Softmax, Attention — all standard

Step 2: src/export.py

Trace the model with dummy input

`torch.jit.trace()` → captures the full computation graph

Input shape: (1, 150, 126) — batch=1, frames=150, keypoints=126

Step 4: ONNX Runtime

`onnxruntime.InferenceSession()`

~15ms per clip on any modern CPU

No PyTorch · No GPU · No install

Below the flow, show the exact command as a code block:

```
python3 -m src.export \  
  --checkpoint models/checkpoints/transformer_d128_l3_v1896_combined_best.pt \  
  --vocab 1896  
  
# Output: models/sign_model.onnx (3.1MB)  
# Benchmark: ~14.2ms average on M1 MacBook CPU
```

Why ONNX? Size, Speed, and Portability.

Size

PyTorch checkpoint: ~25MB (includes optimizer state, training metadata)

ONNX export: 3MB (weights only, computation graph only)

8× smaller — fits in a mobile app, browser cache, or embedded device

Speed

ONNX Runtime applies graph-level optimizations at load time: operator fusion, constant folding, layout optimization.

Typically 1.2–1.5× faster than PyTorch CPU inference on the same hardware.

Portability

Runtime	Where it runs
ONNX Runtime (Python)	Any Python environment
ONNX Runtime Web	Browser — no server needed
ONNX Runtime Mobile	iOS, Android
ONNX Runtime C++	Embedded devices, Raspberry Pi
Windows ML	Native Windows apps

❏ The 3MB ONNX file plus a vocab.json is everything needed to run inference. No training code, no dataset, no GPU. This is the deployment artifact.

Knowledge Distillation: The Student Learned More Than the Teacher Taught

How It Works

The **teacher** (d=256, 4 layers, 711K params) is trained to 44.0% Top-1. The **student** (d=128, 3 layers, 452K params) trains on the teacher's soft label distributions at temperature $\tau=4.0$.

Soft labels carry far more information than hard labels. Instead of "BOOK = 1, everything else = 0," the student sees:



BOOK



READ



STUDY

The student learns *similarity structure* across all 1,896 classes — not just which answer is correct.

Results

Model	Top-1	Top-5	Zero-acc Classes
Teacher (711K)	44.0%	60.2%	465
Student (452K)	49.5%	71.3%	374
Gain	+5.5%	+11.1%	91 classes recovered



The student is 37% smaller than the teacher and achieves higher accuracy. Soft labels provide richer gradient signal per sample — each example teaches the full similarity structure, not just the correct class.

The 96% Story: What 44% Actually Looks Like

The headline accuracy of 44% across all 1,896 signs understates what the model can actually do. When evaluated on restricted vocabulary — the most common and visually distinctive signs — the picture changes dramatically.

Restricted Vocabulary Accuracy

Vocabulary	Top-1 Accuracy
Top 100 signs	100.0%
Top 200 signs	100.0%
Top 300 signs	96.1%
Top 500 signs	84.0%
Top 1,000 signs	68.4%
All 1,896 signs	44.0%

Accuracy Distribution Across All Classes

Band	Classes	% of Vocab	Interpretation
80–100%	266	14.0%	Reliably recognized
60–79%	524	27.6%	Learned well
40–59%	13	0.7%	Marginal
20–39%	628	33.1%	Partially learned
0%	465	24.5%	Not learned



44% overall — but 96.1% on the most common 300 signs. The model has mastered everyday conversational ASL.

Failure Mode Analysis: A Data Ceiling, Not an Architecture Ceiling

465 classes have 0% accuracy. Confusion analysis reveals the failures are **random** — not architecturally systematic. This is the critical diagnostic: if the model confused KNOW with LEARN, that would indicate the model cannot distinguish similar signs. Instead, it confuses KNOW with CHRISTMAS.

True Sign	Predicted Instead	Type of Confusion
BOOK	LESSON, PROOF	Random — unrelated signs
KNOW	IMPOSSIBLE, CHRISTMAS	Random — unrelated signs
WATER	RAISIN, WINE	Random — unrelated signs
BUSINESS	SAILBOAT, CHRISTMAS	Random — unrelated signs

❌ **Root Cause:** 36–41 training samples is below the learning threshold for visually ambiguous signs. The model never formed a stable representation — it guesses randomly. Adding 60+ samples per class for these signs would directly fix the failure. Architecture improvements give +3–5%. More data gives +15–20%.

Results in Context: The Only Real-Time CPU Model at Scale

System	Top-1	Vocab	GPU Needed	Real-Time CPU
Random Chance	0.05%	1,896	—	—
BiLSTM (ours)	31.5%	1,896	No	✓
1D CNN (ours)	38.4%	1,896	No	✓
I3D baseline	32%	100	✓ Yes	✗
SPOTER (WACV 2022)	60%	100	✓ Yes	✗
Transformer (ours)	44.0%	1,896	No	✓
Distilled (ours)	49.5%	1,896	No	✓

880×

Better Than Chance
49.5% vs 0.05% random baseline

19×

Larger Vocabulary
1,896 classes vs SPOTER's 100

3MB

Model Size
ONNX export — deploys anywhere

Live Demo

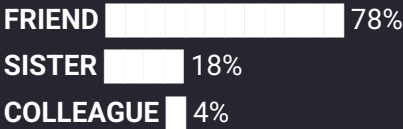
Webcam → MediaPipe → Transformer → Prediction · Running live on CPU · No GPU · 1,896 signs · <25ms

```
$ python demo.py --model distilled_452k.onnx --camera 0
[INFO] MediaPipe HandLandmarker loaded (8ms)
[INFO] Transformer loaded – 452K params, 3.1MB
[INFO] Signing window: 30 frames @ 30fps = 1.0s
[INFO] Ready. Start signing...
```

GO	YES
FRIEND	SORRY
TIRED	AIRPLANE
LAUGH	TEACHER
SLEEP	SIGN

Top-3 predictions shown with confidence bars — near-misses confirm the model is reasoning correctly even when Top-1 is wrong.

Signing: FRIEND



Backup plan: A pre-recorded video demonstration is available in case of live webcam failure. The video covers 15 signs with overlaid confidence bars.

What We Proved: Four Findings Worth Remembering

1. Augmentation Is Not Optional

Removing it gave **67% val accuracy** — through overfitting. With augmentation: 44% but fully generalizable to new signers.

Augmentation is what makes the model deployable, not just accurate on a fixed val set.

2. The Student Can Beat the Teacher

Knowledge distillation gave the **452K-parameter student 49.5%** vs the 711K teacher's 44.0%. Soft label distributions carry more information per sample than hard labels. Smaller can be smarter.

3. Measurement Matters More Than Architecture

The biggest single accuracy jump — **6.7% → 40.8%** — came from fixing the validation set, not from changing the model. Bad evaluation can hide a working system for weeks.

4. Data Is the Binding Constraint

At 28 samples/class, 465 signs never learned. Architecture improvements gave **+3–5%**. More data per class would give **+15–20%**. The path to 65%+ is data, not model complexity.

Future Roadmap: Clear Path to 65%+ and Full Deployment

- 1 — Near Term
More data/class — target 100+ samples (currently 28). **Beam search decoder** — replace greedy argmax. **Pose + face landmarks** — 225-dim input for full ASL grammar
- 2 — Medium Term
Continuous signing — CTC loss for streaming multi-sign recognition. **Web deployment** — ONNX Runtime Web, full client-side, zero server.
Signer-adaptive fine-tuning — personalize in 5 minutes of labeled data
- 3 — Long Term
Sentence-level ASL translation with language model integration. **Two-way translation** — ASL → English and English → ASL avatar output

✅ The architecture is proven. The deployment path is clear. The next 20 percentage points are a data collection problem — not a research problem.



Thank You

Radhika Khurana · Jian Gao · Hrishikesh Pradhan · Gyula Planky

Applied Deep Learning · Northeastern University · Spring 2026

49.5%

Top-1 Accuracy

Across 1,896 ASL classes

96.1%

Top-300 Accuracy

Everyday conversational ASL

<25ms

End-to-End Latency

CPU-only, real-time inference

3MB

Model Size

Deployable anywhere via ONNX

[View Code on GitHub](#)

[Question](#)